



REPÚBLICA DE
MOÇAMBIQUE

MANUAL DE INTRODUÇÃO À PROGRAMAÇÃO COM PYTHON



```
# Python é simples e poderoso  
nome = input("Qual é o seu nome? ")  
print("Olá, " + nome + "!")  
  
for i in range(1, 6):  
    print(i)
```

Ensino Secundário
II-Ciclo



Ensino à Distância



PENSAMENTO
LÓGICO



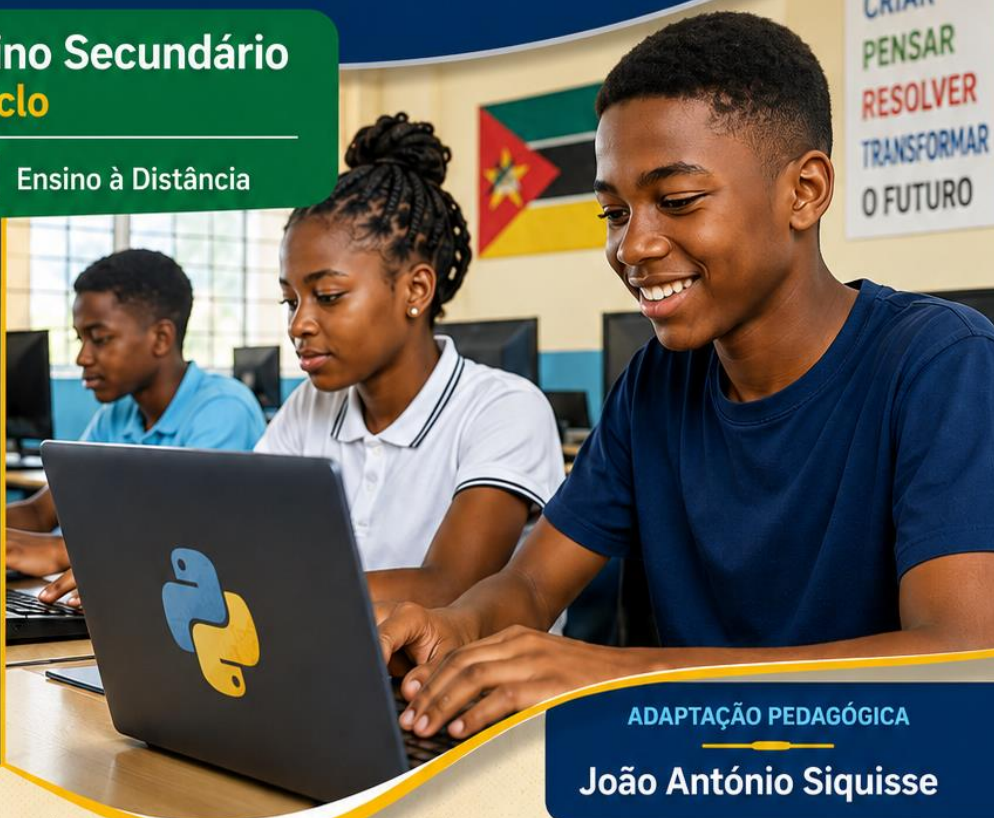
CRIATIVIDADE



RESOLUÇÃO DE
PROBLEMAS



PREPARAÇÃO PARA
O FUTURO



ADAPTAÇÃO PEDAGÓGICA

João António Siquisse



Baseado na obra
"Python para Todos:
Explorando Dados com Python 3"
de Charles Severance



MAPUTO

2026



LICENÇA
CREATIVE COMMONS
ATRIBUIÇÃO-NÃO COMERCIAL-
PARTILHA IGUAL (CC BY-NC-SA)

FICHA TÉCNICA DO RECURSO

Título do recurso:

Manual de Introdução à Programação com Python

Área/Disciplina:

Informática / Programação

Público-alvo:

Estudantes do Ensino Secundário – II Ciclo

Faixa etária recomendada:

15 a 18 anos

Modalidade de utilização:

Ensino presencial, ensino à distância e autoaprendizagem

Objectivo geral de aprendizagem:

Desenvolver competências básicas de programação, pensamento lógico e resolução de problemas através da linguagem Python.

Autor da adaptação pedagógica:

João António Siquisse

Obra original utilizada na adaptação:

Python para Todos: Explorando Dados com Python 3

Charles Severance

Formato do recurso:

PDF e versão editável (.docx)

Língua:

Português

Licença do recurso:

Creative Commons BY-NC-SA 3.0

(Atribuição – Não Comercial – Partilha nos Mesmos Termos)

LICENÇA DE UTILIZAÇÃO

Este recurso educacional aberto encontra-se licenciado sob:

Creative Commons BY-NC-SA 3.0

Isto significa que o recurso pode ser utilizado, partilhado e adaptado para fins educativos, desde que:

- seja atribuída a autoria ao autor da adaptação;
- não seja utilizado para fins comerciais;
- novas versões mantenham a mesma licença aberta.

Este recurso foi preparado para incentivar a partilha de conhecimento e a reutilização pedagógica.

ÍNDICE

CAPÍTULO 1 – VAMOS CONVERSAR SOBRE PROGRAMAÇÃO	9
Introdução	9
1. A Tecnologia Está em Todo Lugar	9
2. O que é Programação?	11
3. O Computador é um Assistente.....	12
4. Por que Aprender Programação?	13
5. Como Funciona um Computador?	15
6. O que é um Programa?	16
7. Resolver Problemas com Programação	17
Resumo do Capítulo	20
CAPÍTULO 2 – PREPARANDO O COMPUTADOR PARA PROGRAMAR	21
Introdução	21
O que vai aprender?	21
1. Antes de instalar o Python	21
2. Verificando se o Python já está instalado	22
3. Transferindo o Python.....	23
4. Instalando o Python.....	23
5. Confirmando a instalação.....	24
6. Abrindo o Python.....	24
7. Criando o primeiro ficheiro Python	25
8. Executando um programa guardado	25
Vamos praticar?	26
CAPÍTULO 3 – CONHECENDO O PYTHON	29
Introdução	29
O que vai aprender?.....	29
1. O que é uma Linguagem de Programação?	29
2. O que é Python?	30
3. Conversando com Python	31
4. O Computador Faz Exactamente o que Mandamos	32
5. O Nosso Primeiro Mini-Programa.....	33
Resumo do Capítulo.....	35

CAPÍTULO 4 – GUARDANDO INFORMAÇÕES COM VARIÁVEIS	36
Introdução	36
1. O que é uma Variável?	36
3. Mostrando o Conteúdo de uma Variável	38
4. Trabalhando com Várias Variáveis.....	39
5. Variáveis de Texto.....	40
6. Variáveis Numéricas	41
7. Construindo um Programa Completo.....	41
Resumo do Capítulo	45
CAPÍTULO 5 – FAZENDO CONTAS COM PYTHON	46
Introdução	46
1. Os Computadores Gostam de Números.....	46
2. A Operação de Soma.....	47
3. A Operação de Subtração.....	48
4. A Operação de Multiplicação.....	48
7. Calculando a Média de Notas.....	50
8. Ordem das Operações	51
9. Construindo uma Calculadora Simples.....	52
Resumo do Capítulo	56
CAPÍTULO 6 – CONVERSANDO COM O UTILIZADOR.....	57
Introdução	57
1. O que é Interacção?.....	58
2. A Função input().....	58
3. Mostrando a Resposta	59
4. Criando um Programa de Saudação.....	60
5. Recebendo Várias Informações.....	61
6. Criando uma Ficha de Aluno	62
Resumo do Capítulo	66
CAPÍTULO 7 – TRABALHANDO COM NÚMEROS E CÁLCULOS.....	68
Introdução	68
O que vai aprender?.....	68
1. Um Pequeno Problema.....	68
2. Transformando Texto em Número	69

3. Calculando a Idade no Próximo Ano.....	69
4. Criando uma Calculadora de Soma	70
5. Calculando a Média de um Aluno	70
6. Aplicações no Quotidiano	71
7. Pequeno Projecto.....	72
CAPÍTULO 8 – TOMANDO DECISÕES COM PYTHON.....	76
Introdução	76
1. O que é uma Decisão?.....	76
2. A Instrução if.....	77
3. Operadores de Comparação	78
4. Verificando a Nota de um Aluno	79
5. Utilizando else	80
6. Criando Programas Mais Interessantes	81
7. Verificando Aprovação Escolar	82
8. Decisões no Quotidiano.....	83
Resumo do Capítulo	86
CAPÍTULO 9 – REPETINDO TAREFAS COM PYTHON.....	87
Introdução	87
1. O que é uma Repetição?	88
2. O Ciclo for	88
3. A Função range().....	89
4. Repetindo Mensagens	90
5. Utilizando Variáveis no Ciclo.....	91
6. Contando Alunos.....	92
7. Criando uma Tabuada Simples.....	93
8. Utilizando Repetições no Quotidiano.....	94
Resumo do Capítulo	98
CAPÍTULO 10 – TRABALHANDO COM LISTAS	99
Introdução	99
O que vai aprender?.....	99
1. O que é uma Lista?	99
2. Criando a Primeira Lista	100
3. Mostrar uma Lista	101

4. Acedendo aos Elementos	102
5. Adicionando Novos Elementos.....	103
6. Percorrendo uma Lista	104
7. Utilizando Listas no Quotidiano	105
8. Criando um Programa Completo.....	106
Resumo do Capítulo	109
CAPÍTULO 11 – O MEU PRIMEIRO PROJECTO EM PYTHON.....	110
Introdução	110
1. O que é um Projecto?	111
Projecto 1 – Ficha Electrónica do Aluno.....	111
Projecto 2 – Calculadora de Média	112
Projecto 3 – Verificando Aprovação	113
Projecto 4 – Lista de Disciplinas	114
Programa	114
Projecto 5 – Saudação Personalizada	115
Actividade Final.....	117
Desafio Final.....	118
Resumo do Capítulo.....	118
GLOSSÁRIO	119
Algoritmo	119
Aplicação.....	119
CPU (Unidade Central de Processamento)	119
Dados	119
Erro	119
Função.....	119
Hardware	120
Input.....	120
Linguagem de Programação.....	120
Lista.....	120
Memória	120
Output	120
Programa	120
Programação	121

Python	121
Variável.....	121
RESPOSTAS DOS EXERCÍCIOS	122
Capítulo 1 – Vamos Conversar sobre Programação	122
Capítulo 2 – Conhecendo o Python.....	122
Capítulo 3 – Guardando Informações com Variáveis	122
Capítulo 4 – Fazendo Contas com Python	122
Capítulo 5 – Conversando com o Utilizador	123
Capítulo 7 – Tomando Decisões com Python.....	123
Capítulo 8 – Repetindo Tarefas com Python.....	123
Capítulo 9 – Trabalhando com Listas	123
Capítulo 11 – O Meu Primeiro Projecto em Python	123
BIBLIOGRAFIA.....	124
Obra principal utilizada na adaptação	124
Recursos complementares	124
CRÉDITOS.....	124
Autor da obra original	124
Adaptação pedagógica.....	124
LICENÇA DO REA DERIVADO	126
Recurso Educacional Aberto Derivado.....	126
Creative Commons Atribuição-NãoComercial-PartilhaIgual 3.0	126
GUIA DE REUTILIZAÇÃO DO RECURSO	127

SOBRE ESTA ADAPTAÇÃO

Este recurso educacional foi adaptado a partir da obra *Python para Todos: Explorando Dados com Python 3*, de Charles Severance.

A versão original encontrava-se em formato PDF não editável, apresentando conteúdo extenso e linguagem orientada a um público geral e académico.

Com o objectivo de tornar o recurso mais adequado ao contexto educativo moçambicano, foi realizada uma adaptação pedagógica, procurando melhorar a acessibilidade, a reutilização e a adequação didáctica para estudantes do Ensino Secundário.

Durante o processo de adaptação, foram realizadas as seguintes melhorias:

- reorganização do conteúdo em capítulos progressivos;
- simplificação da linguagem técnica;
- inclusão de exemplos contextualizados à realidade moçambicana;
- introdução de actividades práticas e exercícios de revisão;
- produção de versão editável para futuras adaptações;
- melhoria da organização pedagógica do conteúdo;
- preparação do recurso para reutilização em diferentes contextos educativos.

Este recurso foi desenvolvido seguindo princípios de Recursos Educacionais Abertos (REA), promovendo acesso, adaptação, partilha e reutilização para fins educativos.

CAPÍTULO 1 – VAMOS CONVERSAR SOBRE PROGRAMAÇÃO

Introdução

Caro(a) aluno(a),

Bem-vindo(a) ao mundo da programação!

Antes de aprendermos a escrever programas, vamos observar algo interessante.

Já utilizou um telemóvel para enviar mensagens?

Já assistiu a vídeos na Internet?

Já utilizou uma calculadora para resolver exercícios de Matemática?

Já levantou dinheiro num ATM?

Todos estes equipamentos realizam tarefas aparentemente inteligentes. Mas será que eles pensam como as pessoas?

A resposta é não.

Os computadores, telemóveis e tablets apenas executam instruções previamente escritas por programadores.

Essas instruções formam aquilo que chamamos programa.

Neste capítulo, vai descobrir o que é programação, por que razão ela é importante e como os computadores realizam tarefas.

Este capítulo foi adaptado do Capítulo 1 do livro *Python para Todos*, de Charles Severance.

O que vai aprender?

Ao terminar este capítulo deverá ser capaz de:

- ✓ Explicar o que é programação;
- ✓ Explicar o que é um programa;
- ✓ Reconhecer situações onde a programação está presente;
- ✓ Identificar os principais componentes de um computador;
- ✓ Compreender a importância da programação.

1. A Tecnologia Está em Todo Lugar

Observe as imagens.

Figura 1 – A tecnologia no nosso dia-a-dia



Legenda: Estudantes utilizando dispositivos digitais durante actividades de aprendizagem. A imagem demonstra como a tecnologia e a programação estão presentes no quotidiano escolar.

Agora responda:

Pense um pouco

1. Quantos telemóveis existem na sua casa?
2. Existe algum computador na sua escola?
3. Já utilizou uma calculadora?
4. Já assistiu a vídeos num telemóvel?

Provavelmente respondeu "sim" a várias destas perguntas.

Mas já pensou como estes equipamentos conseguem funcionar?

Eles funcionam porque possuem programas.

Sem programas, seriam apenas equipamentos electrónicos sem utilidade.

2. O que é Programação?

Imagine que deseja ensinar um colega a preparar chá.

Talvez lhe dê as seguintes instruções:

1. Ferver água;
2. Colocar o chá numa chávena;
3. Adicionar água quente;
4. Esperar alguns minutos;
5. Servir.

Se os passos forem seguidos correctamente, o resultado será um chá pronto para beber.

Na programação acontece algo semelhante.

O computador recebe instruções e executa-as pela ordem indicada.

Segundo o livro *Python para Todos*, programar consiste em escrever instruções para que o computador execute tarefas.

Definição

Programação é o processo de criação de instruções que dizem ao computador o que deve fazer.

Curiosidade

Quando utiliza uma aplicação de mensagens, existem milhares de instruções a funcionar em segundo plano.

Todas essas instruções foram escritas por programadores.

Figura 2 – Seguindo instruções



Legenda: A imagem representa uma sequência organizada de acções, demonstrando que os computadores executam tarefas a partir de instruções definidas previamente.

3. O Computador é um Assistente

O autor do livro apresenta uma comparação muito interessante.

Imagine que o computador é um assistente pessoal.

Esse assistente está constantemente a perguntar:

"O que deseja que eu faça agora?"

Quando recebe instruções, executa-as imediatamente.

Mas existe um problema.

O computador só compreende instruções muito precisas.

Por isso os programadores precisam aprender linguagens de programação.

Exemplo

Se disser ao computador:

1. Mostrar uma mensagem;
2. Fazer uma conta;
3. Guardar uma informação;

ele consegue executar essas tarefas.

Mas apenas se receber instruções claras.

4. Por que Aprender Programação?

Talvez esteja a perguntar:

Professor, para que serve aprender programação?

A programação ajuda-nos a resolver problemas.

Imagine uma escola com 3 000 alunos.

Como descobrir rapidamente:

1. Quantos alunos existem?
2. Quantas raparigas estão matriculadas?
3. Qual é a média das notas?

Fazer isso manualmente seria muito demorado.

Um programa pode efectuar esses cálculos em poucos segundos.

Onde encontramos programação?

Educação

1. Gestão de notas;
2. Matrículas;
3. Plataformas de ensino.

Saúde

1. Registo de pacientes;
2. Marcação de consultas;
3. Sistemas hospitalares.

Agricultura

1. Previsão meteorológica;

2. Gestão de culturas;
3. Sistemas de irrigação.

Comércio

1. Facturação;
2. Gestão de stock;
3. Controlo de vendas.

Bancos

1. ATM;
2. Aplicações bancárias;
3. Transferências electrónicas.

Figura 3 – Programação em diferentes áreas



Legenda: A programação está presente em diferentes sectores profissionais, permitindo automatizar tarefas e resolver problemas em várias áreas da sociedade.

5. Como Funciona um Computador?

Por fora, normalmente vemos:

1. Monitor;
2. Teclado;
3. Rato;
4. Colunas.

Mas dentro do computador existem vários componentes importantes.

Segundo o livro *Python para Todos*, os principais são:

1. CPU;
2. Memória principal;
3. Memória secundária;
4. Dispositivos de entrada;
5. Dispositivos de saída.

CPU

A CPU é conhecida como o cérebro do computador.

É responsável por executar as instruções dos programas.

O livro explica que a CPU está constantemente a perguntar:

"O que faço a seguir?"

Memória Principal

Guarda temporariamente as informações necessárias para executar programas.

Quando o computador é desligado, estas informações desaparecem.

Memória Secundária

Guarda informações permanentemente.

Exemplos:

1. SSD;
2. Disco rígido;
3. Pen drive.

Dispositivos de Entrada

Permitem introduzir informações.

Exemplos:

1. Teclado;
2. Rato;
3. Microfone.

Dispositivos de Saída

Permitem apresentar resultados.

Exemplos:

1. Monitor;
2. Impressora;
3. Colunas.

Figura 4 – Componentes de um computador



Legenda: Principais componentes de um sistema computacional.

6. O que é um Programa?

Já sabemos o que é programação.

Mas o que é exactamente um programa?

Um programa é um conjunto de instruções organizadas para realizar uma tarefa.

Exemplos de programas

1. WhatsApp;
2. Calculadora;
3. Navegador Web;
4. Jogo electrónico;
5. Sistema escolar.

Todos estes exemplos são programas.

Um programa muito simples

Observe:

```
print("Olá!")
```

Quando executado, o computador apresenta:

Olá!

Este pequeno conjunto de instruções já constitui um programa.

Figura 5 – O primeiro programa



Legenda: O primeiro passo para aprender programação é escrever pequenos programas.

7. Resolver Problemas com Programação

Uma das principais funções da programação é resolver problemas.

Observe alguns exemplos:

Problema	Solução
Calcular médias dos alunos	Criar um programa
Organizar notas	Criar um programa
Contar livros da biblioteca	Criar um programa
Controlar vendas	Criar um programa

Por isso, a programação ajuda-nos a:

- ✓ Poupar tempo;
- ✓ Organizar informações;
- ✓ Evitar erros;
- ✓ Automatizar tarefas repetitivas.

Actividades

Actividade 1

Observe os seguintes equipamentos:

1. Telemóvel;
2. ATM;
3. Computador;
4. Tablet.

Escolha um dos equipamentos apresentados e explique, em duas ou três frases, por que razão o utiliza com maior frequência no seu quotidiano.

Actividade 2

Identifique cinco situações do seu quotidiano onde acredita existir programação.

Actividade 3

Indique:

- a) Um dispositivo de entrada.
- b) Um dispositivo de saída.

Actividade 4

Explique, por palavras suas, o que é programação.

Vamos verificar o que aprendeu?

1. O que é programação?

- a) Um equipamento electrónico
- b) Um processo de criação de instruções
- c) Um tipo de computador
- d) Um programa de televisão

2. O que é um programa?

- a) Um teclado
- b) Um conjunto de instruções
- c) Um monitor
- d) Uma impressora

3. Qual é o cérebro do computador?

- a) Monitor
- b) Impressora
- c) CPU
- d) Teclado

4. Qual é um dispositivo de entrada?

- a) Monitor
- b) Impressora
- c) Teclado
- d) Coluna

5. A programação ajuda a:

- a) Criar problemas
- b) Resolver problemas

- c) Apagar ficheiros
- d) Danificar computadores

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Programação é o processo de criação de instruções para computadores;
2. Os computadores executam tarefas através de programas;
3. Um programa é um conjunto de instruções organizadas;
4. A CPU é responsável pela execução das instruções;
5. Existem dispositivos de entrada e de saída;
6. A programação ajuda a resolver problemas do quotidiano.

Parabéns!

Acabou de concluir o primeiro capítulo do manual.

No próximo capítulo, vamos conhecer a linguagem **Python** e escrever o nosso primeiro programa.

CAPÍTULO 2 – PREPARANDO O COMPUTADOR PARA PROGRAMAR

Introdução

Caro(a) aluno(a),

No capítulo anterior aprendemos que os computadores executam programas criados por programadores.

Agora surge uma pergunta importante.

Como podemos escrever e executar os nossos próprios programas?

Antes de começarmos a programar, precisamos preparar o computador.

Para isso, vamos utilizar o **Python**, uma linguagem de programação gratuita, simples de aprender e utilizada em escolas, universidades e empresas de todo o mundo.

Neste capítulo aprenderá a verificar se o Python já está instalado, a instalá-lo caso seja necessário e a executar o seu primeiro programa.

Prepare-se! Depois deste capítulo o seu computador estará pronto para o acompanhar durante toda a aprendizagem da programação.

O que vai aprender?

Ao terminar este capítulo deverá ser capaz de:

- ✓ Verificar se o Python já está instalado no computador;
- ✓ Instalar correctamente o Python no Windows;
- ✓ Confirmar que a instalação foi concluída com sucesso;
- ✓ Executar o primeiro programa em Python;
- ✓ Compreender a importância do ambiente de programação.

1. Antes de instalar o Python

Muitos computadores já possuem o Python instalado.

Antes de efectuar qualquer instalação devemos confirmar essa situação.

Assim evitamos instalar novamente um programa que já existe no computador.

2. Verificando se o Python já está instalado

Passo 1

Prima simultaneamente as teclas:

Win + R

Será apresentada a janela **Executar**.

Passo 2

Na caixa de texto escreva:

`cmd`

Depois prima **Enter**.

Será aberta a janela da Linha de Comandos (Prompt de Comandos).

Passo 3

Escreva o seguinte comando:

`python --version`

e prima **Enter**.

Se aparecer uma mensagem semelhante a esta:

Python 3.14.3

isso significa que o Python já está instalado.

Cada computador poderá apresentar um número de versão diferente.

Por exemplo:

Python 3.12.8

ou

Python 3.13.7

Todas estas versões pertencem à família Python 3 e são adequadas para acompanhar este manual.

Se aparecer uma mensagem de erro...

Se surgir uma mensagem semelhante a:

'python' não é reconhecido como um comando interno ou externo.

não se preocupe.

Significa apenas que o Python ainda não está instalado ou não foi configurado correctamente.

Nos passos seguintes vamos efectuar a instalação.

3. Transferindo o Python

Abra o navegador de Internet.

Aceda ao sítio oficial do Python.

Na página principal encontrará um botão semelhante a:

Download Python 3.x.x

Clique nesse botão.

O ficheiro de instalação será transferido automaticamente para o computador.

Importante

Utilize sempre o sítio oficial do Python para efectuar a instalação. Assim garante que está a utilizar uma versão segura e actualizada.

4. Instalando o Python

Quando terminar a transferência do ficheiro:

1. Abra o instalador.

Aparecerá uma janela semelhante à seguinte:

☐ Install launcher

☐ Add Python to PATH

Install Now

Customize Installation

Muito importante

Antes de clicar em **Install Now**, marque a opção:

☒ **Add Python to PATH**

Esta opção permite executar o Python directamente a partir da Linha de Comandos.

Se não for seleccionada, alguns programas poderão não encontrar o Python correctamente.

Depois clique em:

Install Now

A instalação poderá demorar alguns minutos.

Quando terminar aparecerá a mensagem:

Setup was successful

Clique em:

Close

A instalação está concluída.

5. Confirmando a instalação

Abra novamente a Linha de Comandos.

Escreva:

```
python --version
```

Se surgir uma versão do Python, a instalação foi realizada com sucesso.

Agora verifique também o programa responsável pela instalação de bibliotecas.

Escreva:

```
pip --version
```

Se aparecer uma mensagem semelhante a:

```
pip 25.x from ...
```

está tudo pronto.

6. Abrindo o Python

Na Linha de Comandos escreva:

```
python
```

Aparecerá algo semelhante a:

```
Python 3.14.3
```

```
>>>
```

O símbolo:

```
>>>
```

indica que o Python está pronto para receber instruções.

Escreva:

```
print("Olá, Mundo!")
```

Depois prima **Enter**.

Resultado

Olá, Mundo!

Parabéns!

Acabou de executar o seu primeiro programa em Python.

Para sair escreva:

```
exit()
```

ou pressione as teclas:

Ctrl + Z e depois **Enter**.

7. Criando o primeiro ficheiro Python

Em vez de escrever apenas uma instrução de cada vez, normalmente criamos programas guardados em ficheiros.

Abra o **Bloco de Notas**.

Escreva:

```
print("Olá!")
```

```
print("Estou a aprender Python.")
```

```
print("Bem-vindo ao mundo da programação!")
```

Guarde o ficheiro com o nome:

```
primeiro_programa.py
```

A extensão **.py** identifica um programa escrito em Python.

8. Executando um programa guardado

Abra novamente a Linha de Comandos.

Entre na pasta onde guardou o ficheiro.

Por exemplo:

```
cd Desktop
```

ou

```
cd "Área de Trabalho"
```

(depende da configuração do Windows).

Depois execute:

```
python primeiro_programa.py
```

Resultado esperado

Olá!

Estou a aprender Python.

Bem-vindo ao mundo da programação!

Sabia que?

O Python é uma das linguagens de programação mais utilizadas no mundo.

É empregue em áreas como:

- Educação;
- Desenvolvimento Web;
- Inteligência Artificial;
- Ciência;
- Robótica;
- Análise de dados;
- Automatização de tarefas.

Milhões de programadores utilizam Python diariamente para resolver problemas e desenvolver aplicações.

Curiosidade

Durante este manual utilizará frequentemente a Linha de Comandos.

Não tenha receio de utilizá-la.

Ela é uma das ferramentas mais importantes para qualquer programador.

Vamos praticar?

Actividade 1

Verifique se o Python está instalado no seu computador.

Escreva:

```
python --version
```

Anote a versão apresentada.

Actividade 2

Abra o interpretador do Python.

Escreva:

```
print("Olá!")
```

Observe o resultado.

Actividade 3

Crie um ficheiro chamado:

```
boas_vindas.py
```

Escreva o seguinte programa:

```
print("Bem-vindo!")
```

```
print("Estou a aprender Python.")
```

Execute-o e observe o resultado.

Actividade 4

Altere o programa anterior para apresentar:

- o seu nome;
- a sua escola;
- a sua província.

Vamos verificar o que aprendeu?

1. Qual comando permite verificar a versão instalada do Python?

- a) python
- b) python --version
- c) version
- d) pip

2. Qual opção deve ser seleccionada durante a instalação?

- a) Install Launcher
- b) Add Python to PATH
- c) Cancel

d) Restart

3. Qual é a extensão dos programas escritos em Python?

a) .txt

b) .exe

c) .py

d) .docx

4. Qual comando apresenta uma mensagem no ecrã?

a) show()

b) print()

c) display()

d) echo()

5. Qual símbolo indica que o interpretador Python está pronto para receber comandos?

a)- ###

b)- >>>

c)- \$\$

d)- ***

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Antes de instalar o Python é importante verificar se ele já existe no computador.
2. O Python pode ser transferido gratuitamente a partir do seu sítio oficial.
3. Durante a instalação deve ser seleccionada a opção **Add Python to PATH**.
4. O comando `python --version` permite verificar a versão instalada.
5. O comando `print()` apresenta mensagens no ecrã.
6. Os programas Python são guardados em ficheiros com a extensão **.py**.
7. É possível executar programas através da Linha de Comandos.

Parabéns!

O seu computador está preparado para acompanhar todas as actividades deste manual.

No próximo capítulo, vamos conhecer melhor a linguagem Python e começar a escrever programas cada vez mais interessantes.

CAPÍTULO 3 – CONHECENDO O PYTHON

Introdução

Caro(a) aluno(a),

No capítulo anterior aprendemos que os computadores precisam de instruções para realizar tarefas.

Mas surge uma questão importante:

Como podemos comunicar com um computador?

Será que podemos falar com ele utilizando Português?

Ou será que precisamos de uma linguagem especial?

A resposta é simples.

Os computadores utilizam linguagens de programação.

Uma dessas linguagens chama-se **Python**.

Neste capítulo, vamos conhecer o Python e escrever o nosso primeiro programa.

Prepare-se, porque vai dar os seus primeiros passos como programador!

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Explicar o que é Python;
- ✓ Explicar o que é uma linguagem de programação;
- ✓ Reconhecer algumas características do Python;
- ✓ Escrever o primeiro programa;
- ✓ Executar comandos simples.

1. O que é uma Linguagem de Programação?

Imagine que um turista chega a Moçambique e não fala Português.

Se ninguém compreender a língua dele, será difícil comunicar.

Com os computadores acontece algo semelhante.

Os computadores não compreendem Português nem Inglês da mesma forma que as pessoas.

Por isso foram criadas linguagens especiais chamadas **linguagens de programação**.

Uma linguagem de programação permite que os seres humanos comuniquem com os computadores.

Exemplos de Linguagens de Programação

Existem muitas linguagens de programação.

Algumas das mais conhecidas são:

1. Python;
2. Java;
3. JavaScript;
4. C++;
5. C#.

Cada uma possui características próprias.

Neste manual vamos utilizar Python.

Figura 1 – Diferentes linguagens de programação



Legenda: Existem várias linguagens de programação, cada uma com características específicas.

2. O que é Python?

Python é uma linguagem de programação criada por Guido van Rossum.

Actualmente é uma das linguagens mais utilizadas no mundo.

Segundo o livro *Python para Todos*, Python foi escolhida porque possui uma sintaxe simples e fácil de compreender.

Por essa razão é muito utilizada no ensino da programação.

Onde o Python é utilizado?

Python é utilizado em:

1. Educação;
2. Ciência;
3. Inteligência Artificial;
4. Desenvolvimento Web;
5. Análise de dados;
6. Automação de tarefas.

Mesmo aplicações muito conhecidas utilizam Python em alguns dos seus serviços.

Sabia que?

Muitos cientistas utilizam Python para analisar dados e realizar pesquisas.

3. Conversando com Python

Imagine que encontra uma pessoa pela primeira vez.

Provavelmente dirá:

Olá!

Também podemos ensinar o computador a dizer "Olá".

Observe:

```
print("Olá!")
```

Quando executamos este comando, o computador apresenta:

Olá!

Acabou de escrever o seu primeiro programa.

Parabéns!

Como funciona?

A palavra:

```
print
```

significa:

mostrar algo no ecrã.

O texto colocado entre aspas será apresentado ao utilizador.

Exemplo 1

```
print("Bom dia!")
```

Resultado:

Bom dia!

Exemplo 2

```
print("Bem-vindo ao mundo da programação!")
```

Resultado:

Bem-vindo ao mundo da programação!

Figura 2 – O primeiro programa



Legenda: O primeiro programa geralmente apresenta uma mensagem simples.

4. O Computador Faz Exactlymente o que Mandamos

Uma característica importante dos computadores é a seguinte:

Eles fazem exactamente aquilo que lhes pedimos.

Nem mais.

Nem menos.

Se a instrução estiver correcta, o resultado será correcto.

Se existir um erro, o computador poderá apresentar uma mensagem de erro.

Exemplo

Observe:

```
print("Olá")
```

Funciona correctamente.

Mas se escrevermos:

```
prin("Olá")
```

Python não compreenderá a instrução.

Por isso devemos escrever os comandos correctamente.

Curiosidade

Os erros fazem parte da aprendizagem da programação.

Até os programadores profissionais cometem erros.

5. O Nosso Primeiro Mini-Programa

Vamos criar um programa com várias mensagens.

```
print("Olá!")
```

```
print("O meu nome é Ana.")
```

```
print("Estou a aprender Python.")
```

Resultado:

Olá!

O meu nome é Ana.

Estou a aprender Python.

Observe que cada instrução aparece numa linha diferente.

Figura 3 – Estudante a programar



Legenda: A prática é fundamental para aprender programação.

Vamos praticar?

Actividade 1

Escreva um programa que apresente:

Olá!

Sou estudante.

Actividade 2

Escreva, execute e teste um programa que apresente no ecrã o nome da sua escola.

Actividade 3

Escreva três mensagens diferentes utilizando print().

Vamos verificar o que aprendeu?

1. O que é Python?

- a) Um computador
- b) Uma linguagem de programação
- c) Um sistema operativo
- d) Um navegador

2. Qual comando apresenta uma mensagem?

- a) show()
- b) print()
- c) input()
- d) display()

3. O que aparece no ecrã?

print("Moçambique")

- a) Python
- b) Erro
- c) Moçambique
- d) Computador

4. Python é considerado:

- a) Difícil
- b) Antigo
- c) Simples para iniciantes
- d) Um equipamento

5. Os computadores:

- a) Interpretam emoções
- b) Fazem exactamente o que lhes mandamos
- c) Pensam como seres humanos
- d) Funcionam sem instruções

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Python é uma linguagem de programação;
2. As linguagens de programação permitem comunicar com os computadores;
3. O comando `print()` apresenta mensagens no ecrã;
4. Os computadores executam exactamente as instruções recebidas;
5. Os erros fazem parte do processo de aprendizagem.

Parabéns!

Já escreveu os seus primeiros programas em Python.

No próximo capítulo vamos aprender a guardar informações utilizando **variáveis**. 🚀

Este capítulo já tem dimensão semelhante a um capítulo de manual escolar. O próximo (Variáveis) será ainda mais prático, pois os alunos começarão a criar programas que armazenam nomes, idades, escolas e outras informações próximas da sua realidade.

CAPÍTULO 4 – GUARDANDO INFORMAÇÕES COM VARIÁVEIS

Introdução

Caro(a) aluno(a),

No capítulo anterior aprendeu a utilizar o comando `print()` para apresentar mensagens no ecrã.

Mas imagine a seguinte situação.

O director da escola pede-lhe para guardar:

1. O nome de um aluno;
2. A idade;
3. A turma;
4. A nota de Matemática.

Será que podemos escrever tudo directamente no programa?

Sim, podemos.

Mas existe uma forma melhor e mais organizada.

Para isso utilizamos **variáveis**.

Neste capítulo, vai aprender a guardar informações na memória do computador e a utilizá-las nos seus programas.

Prepare-se para dar mais um passo no mundo da programação!

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Explicar o que é uma variável;
- ✓ Criar variáveis simples;
- ✓ Guardar textos e números;
- ✓ Apresentar valores guardados;
- ✓ Utilizar variáveis em programas simples.

1. O que é uma Variável?

Imagine uma caixa onde pode guardar objectos.

Numa caixa pode guardar livros.

Noutra pode guardar cadernos.

Noutra pode guardar lápis.

As variáveis funcionam de forma semelhante.

Uma variável é um espaço na memória do computador utilizado para guardar informações.

Definição

Variável é um local onde o computador guarda dados para serem utilizados posteriormente.

Exemplo do quotidiano

Imagine uma ficha escolar.

Nela encontramos informações como:

1. Nome;
2. Idade;
3. Turma;
4. Escola.

O computador também precisa de guardar estas informações.

Para isso utilizamos variáveis.

Figura 1 – Variáveis como caixas



Legenda: Uma variável pode ser comparada a uma caixa onde guardamos informações.

2. Criando a Primeira Variável

Observe o exemplo:

```
nome = "Ana"
```

O que aconteceu?

O computador criou uma variável chamada:

```
nome
```

e guardou nela o valor:

```
"Ana"
```

Como ler esta instrução?

Podemos ler assim:

A variável nome recebe o valor Ana.

Outro exemplo

```
idade = 14
```

Neste caso:

1. variável: idade
2. valor: 14

Sabia que?

O símbolo:

=

não significa exactamente "igual".

Na programação significa:

recebe o valor.

3. Mostrando o Conteúdo de uma Variável

Depois de guardar uma informação, podemos mostrá-la.

Observe:

```
nome = "Ana"
```

```
print(nome)
```

Resultado:

Ana

Outro exemplo

```
idade = 14
```

```
print(idade)
```

Resultado:

14

Figura 2 – Guardar e mostrar informação



Legenda: As variáveis permitem guardar e apresentar informações.

4. Trabalhando com Várias Variáveis

Podemos criar várias variáveis no mesmo programa.

Exemplo:

```
nome = "Carlos"
```

```
idade = 15
```

```
turma = "8ª Classe"
```

Agora vamos mostrar os valores:

```
print(nome)
```

```
print(idade)
```

```
print(turma)
```

Resultado:

Carlos

15

8ª Classe

Exemplo Moçambicano

Observe:

```
provincia = "Sofala"
```

```
cidade = "Beira"
```

Mostrar:

```
print(provincia)
```

```
print(cidade)
```

Resultado:

Sofala

Beira

5. Variáveis de Texto

Quando guardamos palavras ou frases utilizamos aspas.

Exemplos:

```
nome = "Maria"
```

```
escola = "Escola Secundária de Xai-Xai"
```

As aspas indicam que estamos a trabalhar com texto.

Mais exemplos

```
pais = "Moçambique"
```

```
disciplina = "Matemática"
```

6. Variáveis Numéricas

Também podemos guardar números.

Exemplo:

```
idade = 13
```

```
nota = 15
```

```
alunos = 42
```

Observe que os números não utilizam aspas.

Comparação

Texto:

```
nome = "Paulo"
```

Número:

```
idade = 15
```

Curiosidade

Python distingue textos de números.

Por isso devemos ter atenção ao utilizar aspas.

7. Construindo um Programa Completo

Vamos criar um pequeno programa.

```
nome = "João"
```

```
idade = 14
```

```
escola = "Escola Secundária de Chókwè"
```

```
print(nome)
```

```
print(idade)
```

```
print(escola)
```

Resultado:

```
João
```

```
14
```

```
Escola Secundária de Chókwè
```

Parabéns!

Acabou de criar um pequeno registo electrónico.

Figura 3 – Ficha electrónica simples



Legenda: As variáveis são utilizadas para guardar informações em sistemas informáticos.

Erros Comuns

Ao trabalhar com variáveis, os iniciantes costumam cometer alguns erros.

Erro 1 – Esquecer as aspas

Incorrecto:

```
nome = Ana
```

Correcto:

```
nome = "Ana"
```

Erro 2 – Escrever o nome errado

Incorrecto:

```
nome = "Ana"
```

```
print(nom)
```

Correcto:

```
nome = "Ana"
```

```
print(nome).
```

Erro 3 – Confundir maiúsculas e minúsculas

Observe:

Nome = "Ana"

é diferente de:

nome = "Ana"

Para Python, são variáveis diferentes.

Vamos praticar?

Actividade 1

No ambiente Python, crie as variáveis indicadas abaixo, execute o programa e observe o resultado apresentado no ecrã:

1. nome
2. idade
3. escola

Apresente os valores utilizando print().

Actividade 2

Crie variáveis para:

1. província
2. distrito
3. localidade

Mostre os valores no ecrã.

Actividade 3

Crie uma ficha simples contendo:

1. nome
2. idade
3. turma
4. disciplina favorita

Desafio

Tente criar um programa que apresente:

Nome: Amélia

Idade: 13

Turma: 7ª Classe

Utilize variáveis.

Vamos verificar o que aprendeu?

1. O que é uma variável?

- a) Um monitor
- b) Um espaço para guardar informações
- c) Um programa
- d) Um teclado

2. Qual instrução cria uma variável?

- a)
`print("Ana")`
- b)
`nome = "Ana"`
- c)
`input("Ana")`
- d)
`mostrar("Ana")`

3. Qual das opções representa um número?

- a)
`"15"`
- b)
`"idade"`
- c)
`15`

d)

"Moçambique"

4. Qual comando apresenta o conteúdo de uma variável?

a)

mostrar()

b)

input()

c)

print()

d)

valor()

5. Qual é o erro?

nome = Ana

a) Falta um número

b) Falta uma variável

c) Faltam aspas

d) Falta print()

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Uma variável é um espaço utilizado para guardar informações;
2. As variáveis podem armazenar textos e números;
3. O comando print() permite apresentar valores guardados;
4. Os textos devem ser colocados entre aspas;
5. As variáveis ajudam a organizar programas.

Parabéns!

Agora já consegue guardar informações nos seus programas.

No próximo capítulo vamos aprender a utilizar variáveis para efectuar cálculos matemáticos com Python.

CAPÍTULO 5 – FAZENDO CONTAS COM PYTHON

Introdução

Caro(a) aluno(a),

No capítulo anterior aprendeu a guardar informações utilizando variáveis.

Por exemplo, conseguiu guardar:

1. Nome;
2. Idade;
3. Escola;
4. Turma.

Mas imagine agora que o director da escola deseja calcular a média das notas dos alunos.

Ou que um comerciante pretende somar as vendas do dia.

Ou ainda que um agricultor deseja calcular a quantidade total de sacos colhidos.

Será que o computador consegue ajudar?

Claro que sim!

Uma das principais funções dos computadores é realizar cálculos de forma rápida e precisa.

Neste capítulo, vai aprender a utilizar Python para efectuar operações matemáticas simples.

Prepare-se para transformar o computador numa poderosa calculadora!

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Utilizar números em Python;
- ✓ Realizar operações matemáticas;
- ✓ Utilizar variáveis em cálculos;
- ✓ Resolver problemas simples;
- ✓ Criar pequenos programas matemáticos.

1. Os Computadores Gostam de Números

Os computadores são excelentes a trabalhar com números.

Podem efectuar milhares de cálculos em poucos segundos.

Observe o exemplo:

```
print(5 + 3)
```

Resultado:

8

O computador efectuou a soma e apresentou o resultado.

Figura 1 – Python como calculadora



Legenda: Python pode funcionar como uma poderosa calculadora.

2. A Operação de Soma

A soma utiliza o símbolo:

+

Exemplos:

```
print(10 + 5)
```

Resultado:

15

```
print(20 + 30)
```

Resultado:

50

Exemplo do quotidiano

Imagine que João possui:

1. 5 cadernos;

2. recebe mais 3 cadernos.

Quantos cadernos terá agora?

```
print(5 + 3)
```

Resultado:

8

3. A Operação de Subtração

A subtração utiliza o símbolo:

-

Exemplo:

```
print(15 - 4)
```

Resultado:

11

Exemplo do quotidiano

Maria tinha 20 meticais.

Gastou 5 meticais.

Quanto sobrou?

```
print(20 - 5)
```

Resultado:

15

4. A Operação de Multiplicação

A multiplicação utiliza o símbolo:

*

Exemplo:

```
print(6 * 4)
```

Resultado:

24

Exemplo do cotidiano

Uma turma possui 5 filas.

Cada fila tem 6 alunos.

Quantos alunos existem?

```
print(5 * 6)
```

Resultado:

30

5. A Operação de Divisão

A divisão utiliza o símbolo:

/

Exemplo:

```
print(20 / 4)
```

Resultado:

5.0

Exemplo do cotidiano

20 mangas são distribuídas igualmente por 4 alunos.

Quantas mangas recebe cada aluno?

```
print(20 / 4)
```

Resultado:

5.0

Figura 2 – Operações matemáticas



Legenda: Python permite realizar diferentes operações matemáticas.

6. Utilizando Variáveis nos Cálculos

Até agora utilizámos números directamente.

Mas também podemos utilizar variáveis.

Observe:

```
nota1 = 12
```

```
nota2 = 15
```

```
print(nota1 + nota2)
```

Resultado:

27

Outro exemplo

```
idade = 14
```

```
print(idade + 1)
```

Resultado:

15

O programa calculou a idade do próximo ano.

7. Calculando a Média de Notas

Uma das aplicações mais úteis é calcular médias.

Imagine que um aluno obteve:

1. 12 valores;

2. 16 valores.

Vamos calcular a média.

```
nota1 = 12
```

```
nota2 = 16
```

```
media = (nota1 + nota2) / 2
```

```
print(media)
```

Resultado:

14.0

Explicação

Primeiro:

nota1 + nota2

Resultado:

28

Depois:

28 / 2

Resultado:

14

Figura 3 – Calculando médias escolares



Legenda: Os computadores ajudam a calcular médias de forma rápida.

8. Ordem das Operações

Python segue regras matemáticas.

Observe:

```
print(2 + 3 * 4)
```

Resultado:

14

Porquê?

Porque a multiplicação é efectuada primeiro.

Se quisermos alterar a ordem:

```
print((2 + 3) * 4)
```

Resultado:

20

Os parênteses têm prioridade.

Sabia que?

Os parênteses ajudam Python a compreender exactamente o que queremos calcular.

9. Construindo uma Calculadora Simples

Vamos criar um pequeno programa.

```
a = 10
```

```
b = 5
```

```
print(a + b)
```

```
print(a - b)
```

```
print(a * b)
```

```
print(a / b)
```

Resultado:

15

5

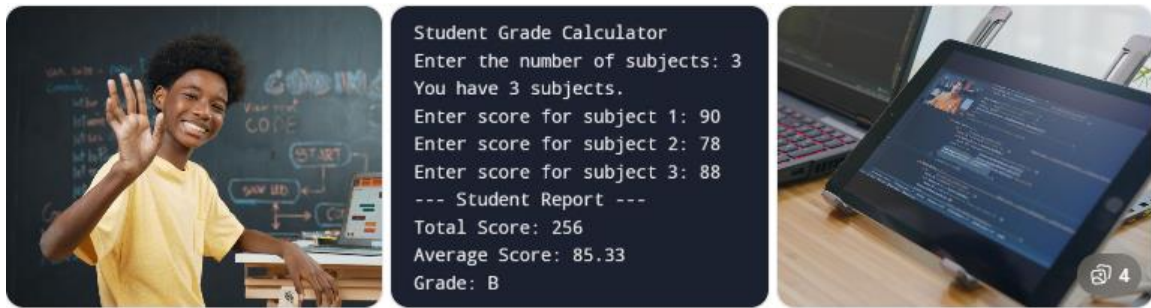
50

2.0

Parabéns!

Acabou de criar uma calculadora muito simples.

Figura 4 – A calculadora do programador



Legenda: Os programadores utilizam computadores para efectuar cálculos automaticamente.

Erros Comuns

Erro 1 – Esquecer um número

Incorrecto:

```
print(10 + )
```

Correcto:

```
print(10 + 5)
```

Erro 2 – Esquecer um parêntese

Incorrecto:

```
print((10 + 5)
```

Correcto:

```
print((10 + 5))
```

Erro 3 – Dividir por zero

Incorrecto:

```
print(10 / 0)
```

Python apresentará um erro.

Vamos praticar?

Actividade 1

Calcule:

1. $12 + 8$
2. $20 - 7$
3. 5×9
4. $36 \div 6$

Actividade 2

Crie duas variáveis:

idade1

idade2

Calcule a soma das idades.

Actividade 3

Crie um programa para calcular a média das notas:

1. 14
2. 16

Actividade 4

Um agricultor colheu:

1. 25 sacos de milho;
2. 18 sacos de feijão.

Calcule a produção total.

Desafio

Crie um programa que calcule:

1. Quantos anos terá daqui a 5 anos.

Exemplo:

idade = 14

Resultado esperado:

19

Vamos verificar o que aprendeu?

1. Qual símbolo representa a soma?

- a) *
- b) /
- c) +
- d) -

2. Qual símbolo representa a multiplicação?

- a) *
- b) +
- c) /
- d) =

3. Qual é o resultado?

`print(10 + 5)`

- a) 5
- b) 10
- c) 15
- d) 20

4. Qual é o resultado?

`print(20 / 4)`

- a) 4
- b) 5
- c) 10
- d) 20

5. Para calcular médias utilizamos:

- a) Apenas texto
- b) Operações matemáticas
- c) Impressoras
- d) Monitores

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Python pode realizar cálculos matemáticos;
2. Existem operações de soma, subtracção, multiplicação e divisão;
3. As variáveis podem participar em cálculos;
4. É possível calcular médias automaticamente;
5. Os computadores executam cálculos com rapidez e precisão.

Parabéns!

Agora já consegue utilizar Python para efectuar cálculos.

No próximo capítulo vamos aprender a **receber informações do utilizador**, tornando os programas mais interactivos.

CAPÍTULO 6 – CONVERSANDO COM O UTILIZADOR

Introdução

Caro(a) aluno(a),

Nos capítulos anteriores aprendeu a:

1. Mostrar mensagens utilizando `print()`;
2. Guardar informações em variáveis;
3. Realizar cálculos matemáticos.

Mas existe uma limitação.

Observe o programa abaixo:

```
nome = "Ana"
```

```
print(nome)
```

Este programa mostra sempre o nome **Ana**.

E se quisermos que cada utilizador escreva o seu próprio nome?

E se quisermos perguntar a idade?

Ou a escola onde estuda?

Para isso precisamos de aprender a receber informações.

Neste capítulo, vai aprender a utilizar a função `input()`, que permite ao utilizador comunicar com o programa.

A partir deste momento, os seus programas tornar-se-ão mais interactivos.

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Utilizar a função `input()`;
- ✓ Receber informações do utilizador;
- ✓ Guardar respostas em variáveis;
- ✓ Mostrar informações recebidas;
- ✓ Criar programas interactivos simples.

1. O que é Interação?

Imagine que um professor faz a seguinte pergunta:

Qual é o seu nome?

O aluno responde:

O meu nome é Carlos.

Isto é uma interação.

Existe uma pergunta e uma resposta.

Os programas também podem fazer perguntas aos utilizadores.

Figura 1 – Comunicação entre pessoas



Legenda: A comunicação envolve perguntas e respostas.

2. A Função input()

Em Python utilizamos a função:

`input()`

para receber informações do utilizador.

Observe:

```
nome = input("Qual é o seu nome? ")
```

Quando o programa é executado, aparece:

Qual é o seu nome?

O utilizador escreve a resposta.

Por exemplo:

Carlos

O valor fica guardado na variável:

nome

Como funciona?

O programa:

1. Mostra a pergunta;
2. Espera uma resposta;
3. Guarda a resposta na variável.

Simples, não é?

3. Mostrando a Resposta

Depois de receber a resposta, podemos apresentá-la.

Observe:

```
nome = input("Qual é o seu nome? ")
```

```
print(nome)
```

Se o utilizador escrever:

Maria

o programa apresenta:

Maria

Outro exemplo

```
cidade = input("Em que cidade vive? ")
```

```
print(cidade)
```

Resposta possível:

Beira

Resultado:

Beira

Figura 2 – Recebendo informações



Legenda: O utilizador pode introduzir informações através do teclado.

4. Criando um Programa de Saudação

Vamos criar um programa mais interessante.

```
nome = input("Qual é o seu nome? ")
```

```
print("Olá!")
```

```
print(nome)
```

Exemplo

Pergunta:

Qual é o seu nome?

Resposta:

João

Resultado:

Olá!

João

Melhorando o programa

```
nome = input("Qual é o seu nome? ")
```

```
print("Bem-vindo ao mundo da programação!")
```

```
print(nome)
```

Sabia que?

Muitas aplicações que utiliza diariamente recebem informações dos utilizadores.

Por exemplo:

1. Formulários escolares;
2. Redes sociais;
3. Aplicações bancárias;
4. Sistemas de matrícula.

5. Recebendo Várias Informações

Podemos fazer mais do que uma pergunta.

Observe:

```
nome = input("Nome: ")
```

```
idade = input("Idade: ")
```

```
escola = input("Escola: ")
```

Agora temos três informações guardadas.

Mostrar os resultados

```
print(nome)
```

```
print(idade)
```

```
print(escola)
```

Exemplo

Resposta do utilizador:

Nome: Amélia

Idade: 13

Escola: Escola Secundária de Chibuto

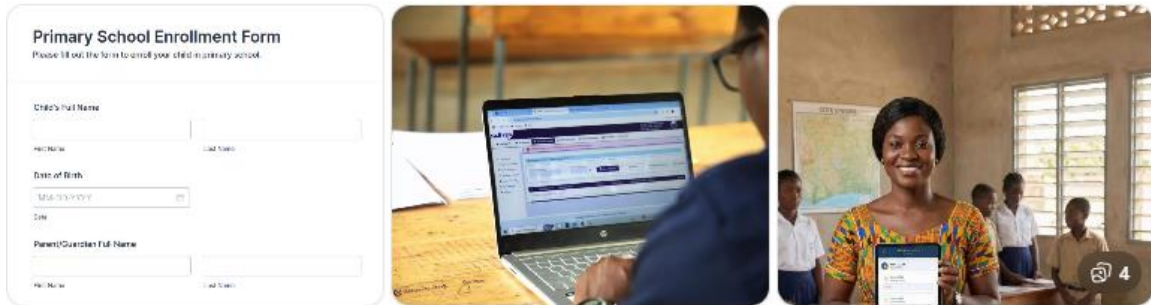
Resultado:

Amélia

13

Escola Secundária de Chibuto

Figura 3 – Ficha electrónica



Legenda: Os sistemas informáticos recolhem informações através de formulários.

6. Criando uma Ficha de Aluno

Vamos construir um pequeno sistema.

```
nome = input("Nome: ")
```

```
idade = input("Idade: ")
```

```
turma = input("Turma: ")
```

Mostrar os resultados:

```
print("----- FICHA DO ALUNO -----")
```

```
print(nome)
```

```
print(idade)
```

```
print(turma)
```

Exemplo

Resposta:

Nome: Carlos

Idade: 14

Turma: 8ª Classe

Resultado:

```
----- FICHA DO ALUNO -----
```

Carlos

14

8ª Classe

Parabéns!

Acabou de criar uma pequena ficha electrónica.

7. Input e Variáveis

Observe:

```
provincia = input("Província: ")
```

O valor digitado ficará guardado em:

```
provincia
```

O mesmo acontece com:

```
distrito = input("Distrito: ")
```

ou

```
escola = input("Escola: ")
```

A função input() e as variáveis trabalham em conjunto.

Aplicação Prática

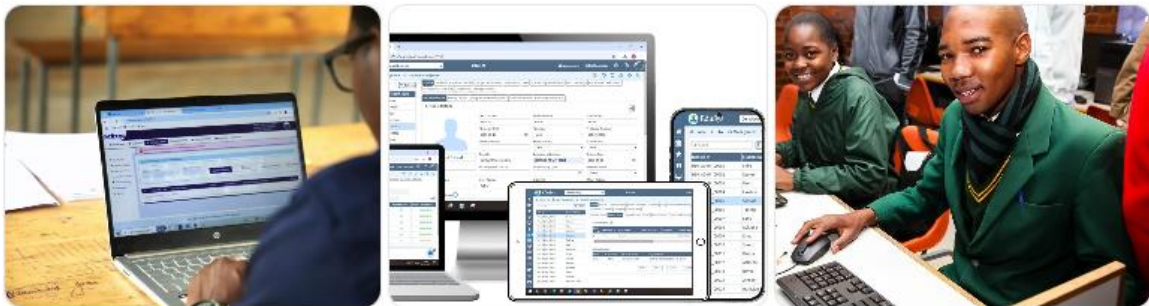
Imagine uma escola que pretende registar:

1. Nome;
2. Idade;
3. Turma;
4. Escola.

Um programa simples pode fazer esse trabalho automaticamente.

É assim que muitos sistemas de informação funcionam.

Figura 4 – Sistema de registo escolar



Legenda: Muitas escolas utilizam sistemas para registar informações dos alunos.

Erros Comuns

Erro 1 – Esquecer os parênteses

Incorrecto:

```
nome = input
```

Correcto:

```
nome = input("Nome: ")
```

Erro 2 – Esquecer as aspas

Incorrecto:

```
nome = input(Nome:)
```

Correcto:

```
nome = input("Nome: ")
```

Erro 3 – Esquecer a variável

Incorrecto:

```
input("Nome: ")
```

Correcto:

```
nome = input("Nome: ")
```

Vamos praticar?

Actividade 1

Crie um programa que pergunte:

1. Nome;
2. Idade.

Mostre as respostas.

Actividade 2

Crie um programa que pergunte:

1. Escola;
2. Turma.

Mostre os resultados.

Actividade 3

Crie uma ficha contendo:

1. Nome;
2. Idade;
3. Escola;
4. Disciplina favorita.

Actividade 4

Crie um programa que pergunte:

1. Província;
2. Distrito.

Apresente as respostas no ecrã.

Desafio

Crie um programa que apresente:

Bem-vindo!

e depois pergunte o nome do utilizador.

No final deve mostrar:

Olá, João!

(substituindo João pelo nome introduzido).

Vamos verificar o que aprendeu?

1. Qual função recebe informações do utilizador?

- a) print()
- b) show()
- c) input()
- d) valor()

2. O que faz a função input()?

- a) Apaga dados
- b) Recebe informações

- c) Faz cálculos
- d) Fecha o programa

3. Qual instrução está correcta?

- a)
`nome = input("Nome: ")`
- b)
`input = nome`
- c)
`print("Nome")`
- d)
`nome("Input")`

4. As respostas recebidas devem ser guardadas em:

- a) Impressoras
- b) Monitores
- c) Variáveis
- d) Teclados

5. Programas interactivos:

- a) Fazem perguntas ao utilizador
- b) Não recebem informações
- c) Apenas mostram mensagens
- d) Não utilizam variáveis

Resumo do Capítulo

Neste capítulo aprendeu que:

1. A função `input()` permite receber informações;
2. As respostas podem ser guardadas em variáveis;
3. Os programas podem fazer perguntas aos utilizadores;
4. É possível criar formulários simples;
5. Muitos sistemas reais utilizam mecanismos semelhantes para recolher dados.

Parabéns!

Agora já consegue criar programas interactivos.

No próximo capítulo vamos aprender a **tomar decisões com Python**, permitindo que os programas escolham diferentes caminhos de acordo com as informações recebidas.

CAPÍTULO 7 – TRABALHANDO COM NÚMEROS E CÁLCULOS

Introdução

Caro(a) aluno(a),

No capítulo anterior aprendeu a receber informações utilizando a função `input()`.

Agora já consegue criar programas que fazem perguntas aos utilizadores.

Mas imagine a seguinte situação.

Um professor pretende calcular a média de um aluno.

Ou um agricultor deseja calcular a quantidade total de sacos colhidos.

Ou ainda um comerciante pretende calcular o valor total das vendas do dia.

Será que o computador consegue ajudar?

Claro que sim!

Neste capítulo vamos aprender a utilizar os dados introduzidos pelos utilizadores para realizar cálculos.

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Receber números através do teclado;
- ✓ Converter textos em números;
- ✓ Realizar cálculos utilizando dados introduzidos pelo utilizador;
- ✓ Construir pequenas calculadoras;
- ✓ Resolver problemas simples do quotidiano.

1. Um Pequeno Problema

Observe o programa:

```
idade = input("Qual é a sua idade? ")
```

```
print(idade)
```

Se o utilizador digitar:

14

o resultado será:

14

Até aqui tudo parece correcto.

Mas observe:

```
idade = input("Qual é a sua idade? ")
```

```
print(idade + 1)
```

Python apresentará um erro.

Porquê?

Porque a função `input()` guarda a resposta como texto.

2. Transformando Texto em Número

Para efectuar cálculos precisamos converter o texto em número.

Utilizamos:

```
int()
```

Exemplo

```
idade = int(input("Qual é a sua idade? "))
```

Agora Python compreende que estamos a trabalhar com um número.

Figura 1 – Introdução de números



Legenda: Os números introduzidos pelo utilizador podem ser utilizados em cálculos.

3. Calculando a Idade no Próximo Ano

Observe:

```
idade = int(input("Qual é a sua idade? "))
```

```
print(idade + 1)
```

Exemplo

Entrada:

14

Resultado:

15

O programa calculou a idade do próximo ano.

4. Criando uma Calculadora de Soma

Vamos criar um programa que soma dois números.

```
numero1 = int(input("Primeiro número: "))
```

```
numero2 = int(input("Segundo número: "))
```

```
resultado = numero1 + numero2
```

```
print(resultado)
```

Exemplo

Entrada:

10

5

Resultado:

15

5. Calculando a Média de um Aluno

Imagine que um aluno obteve:

1. 14 valores;

2. 16 valores.

Vamos calcular a média.

```
nota1 = int(input("Primeira nota: "))
```

```
nota2 = int(input("Segunda nota: "))
```

```
media = (nota1 + nota2) / 2
```

```
print(media)
```

Exemplo

Entrada:

14

16

Resultado:

15.0

Figura 2 – Cálculo de médias



Legenda: Os computadores podem ajudar a calcular médias escolares.

6. Aplicações no Quotidiano

Os cálculos estão presentes em muitas actividades.

Na escola

1. Médias;
2. Estatísticas;
3. Frequências.

Na agricultura

1. Produção total;
2. Quantidade de sementes;
3. Área cultivada.

No comércio

1. Vendas;
2. Lucros;
3. Descontos.

Em casa

1. Despesas;
2. Consumo de água;
3. Consumo de energia.

7. Pequeno Projecto

Vamos criar uma calculadora de produção agrícola.

```
milho = int(input("Sacos de milho: "))
```

```
feijao = int(input("Sacos de feijão: "))
```

```
total = milho + feijao
```

```
print("Produção total:", total)
```

Exemplo

Entrada:

25

18

Resultado:

Produção total: 43

Figura 3 – Agricultura e tecnologia



Legenda: A programação pode ajudar a organizar informações agrícolas.

Vamos praticar?

Actividade 1

Crie um programa que receba dois números e apresente a soma.

Actividade 2

Crie um programa que calcule a sua idade daqui a cinco anos.

Actividade 3

Crie um programa que receba duas notas e calcule a média.

Actividade 4

Crie um programa que receba:

1. número de livros de Português;
2. número de livros de Matemática.

Apresente o total.

Vamos verificar o que aprendeu?

1. O que faz a função `int()`?

- a) Mostra mensagens
- b) Converte texto em número
- c) Repete instruções
- d) Cria listas

2. Qual função recebe informações do utilizador?

- a) print()
- b) range()
- c) input()
- d) if

3. Qual instrução está correcta?

`idade = int(input("Idade: "))`

- a) Correcta
- b) Incorrecta

4. Para calcular médias utilizamos:

- a) Apenas texto
- b) Operações matemáticas
- c) Listas
- d) Ciclos

5. Qual é o resultado?

`print(10 + 5)`

- a) 5
- b) 10
- c) 15
- d) 20

Resumo do Capítulo

Neste capítulo aprendeu que:

1. A função `input()` recebe dados sob a forma de texto;
2. A função `int()` converte texto em números;
3. É possível efectuar cálculos com dados introduzidos pelo utilizador;
4. Os computadores podem calcular médias, somas e totais;
5. A programação ajuda a resolver problemas do quotidiano.

Parabéns!

Agora os seus programas já conseguem receber números e efectuar cálculos.

No próximo capítulo vamos aprender a **tomar decisões com Python utilizando `if` e `else`**.

CAPÍTULO 8 – TOMANDO DECISÕES COM PYTHON

Introdução

Caro(a) aluno(a),

Nos capítulos anteriores aprendeu a:

1. Mostrar mensagens utilizando `print()`;
2. Guardar informações em variáveis;
3. Receber informações utilizando `input()`;
4. Realizar cálculos matemáticos.

Mas imagine a seguinte situação.

Um professor precisa verificar se um aluno foi aprovado.

Se a nota for igual ou superior a 10 valores, o aluno está aprovado.

Caso contrário, está reprovado.

Será que o computador consegue tomar esta decisão?

Sim!

Neste capítulo vai aprender a utilizar estruturas de decisão, permitindo que os programas escolham diferentes caminhos conforme as informações recebidas.

É aqui que os programas começam a tornar-se mais inteligentes.

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Compreender o conceito de decisão;
- ✓ Utilizar a instrução `if`;
- ✓ Comparar valores;
- ✓ Criar programas que tomam decisões;
- ✓ Resolver problemas simples utilizando condições.

1. O que é uma Decisão?

Todos os dias tomamos decisões.

Por exemplo:

1. Se estiver a chover, levo guarda-chuva.

2. Se estiver com fome, vou comer.
3. Se terminar os trabalhos de casa, posso brincar.

Observe que todas estas situações possuem uma condição.

Primeiro verifica-se uma situação.

Depois toma-se uma decisão.

Os computadores também conseguem fazer isso.

Figura 1 – Tomando decisões



Legenda: Todos os dias tomamos decisões com base em condições.

2. A Instrução if

Em Python utilizamos a palavra:

if

para tomar decisões.

A palavra inglesa **if** significa:

se

Exemplo

idade = 15

```
if idade >= 12:
```

```
    print("Pode participar")
```

Resultado:

Pode participar

Como funciona?

O computador verifica:

idade $\geq$ 12

Pergunta:

A idade é maior ou igual a 12?

Resposta:

Sim.

Por isso a mensagem é apresentada.

3. Operadores de Comparação

Para tomar decisões precisamos comparar valores.

Observe os principais operadores.

Operador	Significado
----------	-------------

==	Igual a
!=	Diferente de
>	Maior que
<	Menor que
$\geq$	Maior ou igual a
$\leq$	Menor ou igual a

Exemplos

$10 > 5$

Resultado:

Verdadeiro

$4 > 8$

Resultado:

Falso

Sabia que?

Os computadores trabalham frequentemente com duas respostas:

1. Verdadeiro;
2. Falso.

Estas respostas ajudam-nos a tomar decisões.

4. Verificando a Nota de um Aluno

Imagine que um aluno obteve:

nota = 14

Vamos verificar se foi aprovado.

nota = 14

```
if nota >= 10:
```

```
    print("Aprovado")
```

Resultado:

Aprovado

Outro exemplo

nota = 8

```
if nota >= 10:
```

```
    print("Aprovado")
```

Resultado:

Nenhuma mensagem será apresentada.

Porquê?

Porque a condição não foi satisfeita.

Figura 2 – Avaliação escolar



Legenda: Os computadores podem ajudar a verificar resultados escolares.

5. Utilizando else

Às vezes queremos indicar o que acontece quando a condição não é verdadeira.

Para isso utilizamos:

else

A palavra inglesa **else** significa:

caso contrário

Exemplo

nota = 8

```
if nota >= 10:
```

```
    print("Aprovado")
```

```
else:
```

```
    print("Reprovado")
```

Resultado:

Reprovado

Outro exemplo

idade = 10

```
if idade >= 12:
```

```
    print("Pode participar")
```

else:

```
print("Ainda não pode participar")
```

Resultado:

Ainda não pode participar

6. Criando Programas Mais Interessantes

Agora podemos combinar input() com if.

Observe:

```
idade = int(input("Qual é a sua idade? "))
```

```
if idade >= 18:
```

```
    print("Maior de idade")
```

```
else:
```

```
    print("Menor de idade")
```

O que faz este programa?

1. Pergunta a idade;
2. Recebe a resposta;
3. Verifica a condição;
4. Apresenta o resultado.

Exemplo

Resposta:

20

Resultado:

Maior de idade

Exemplo

Resposta:

14

Resultado:

Menor de idade

Figura 3 – Programa interactivo



Legenda: Os programas podem tomar decisões com base nas respostas dos utilizadores.

7. Verificando Aprovação Escolar

Vamos construir um programa mais útil.

```
nota = int(input("Digite a nota: "))
```

```
if nota >= 10:
```

```
    print("Aprovado")
```

```
else:
```

```
    print("Reprovado")
```

Exemplo

Resposta:

15

Resultado:

Aprovado

Exemplo

Resposta:

7

Resultado:

Reprovado

8. Decisões no Quotidiano

Observe alguns exemplos.

Agricultura

```
if chuva:  
    plantar()
```

Escola

```
if nota >= 10:  
    aprovado
```

Biblioteca

```
if livro_disponivel:  
    emprestar()
```

Comércio

```
if stock > 0:  
    vender()
```

Todos estes exemplos utilizam decisões.

Figura 4 – Decisões em diferentes áreas



Legenda: As decisões estão presentes em muitas aplicações informáticas.

Erros Comuns

Erro 1 – Esquecer os dois pontos

Incorrecto:

```
if nota >= 10
```

Correcto:

```
if nota >= 10:
```

Erro 2 – Utilizar = em vez de ==

Incorrecto:

```
if nota = 10:
```

Correcto:

```
if nota == 10:
```

Erro 3 – Problemas de indentação

Incorrecto:

```
if nota >= 10:
```

```
print("Aprovado")
```

Correcto:

```
if nota >= 10:
```

```
    print("Aprovado")
```

Sabia que?

Os espaços antes das instruções são muito importantes em Python.

Vamos praticar?

Actividade 1

Crie um programa que verifique se um aluno foi aprovado.

Actividade 2

Crie um programa que indique se uma pessoa é maior ou menor de idade.

Actividade 3

Pergunte a idade do utilizador.

Se tiver 12 anos ou mais, apresente:

Pode participar

Caso contrário:

Ainda não pode participar

Actividade 4

Pergunte uma nota.

Apresente:

1. Excelente (16 ou mais);
2. Aprovado (10 a 15);
3. Reprovado (menos de 10).

(Desafio: pesquise como utilizar mais de uma condição.)

Vamos verificar o que aprendeu?

1. Qual palavra utilizamos para tomar decisões?

- a) print
- b) input
- c) if
- d) show

2. O que significa if?

- a) repetir
- b) mostrar
- c) se
- d) guardar

3. O que significa else?

- a) mostrar
- b) caso contrário

- c) repetir
- d) guardar

4. Qual operador significa "maior ou igual a"?

- a) >
- b) <=
- c) >=
- d) ==

5. Qual resultado será apresentado?

nota = 12

```
if nota >= 10:
```

```
    print("Aprovado")
```

- a) Reprovado
- b) Nenhuma mensagem
- c) Aprovado
- d) Erro

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Os programas podem tomar decisões;
2. A instrução if permite verificar condições;
3. A instrução else permite definir uma alternativa;
4. Existem operadores de comparação;
5. É possível criar programas que verificam notas, idades e outras informações.

Parabéns!

Agora os seus programas já conseguem tomar decisões.

No próximo capítulo vamos aprender a **repetir tarefas automaticamente utilizando ciclos**, evitando escrever as mesmas instruções várias vezes.

CAPÍTULO 9 – REPETINDO TAREFAS COM PYTHON

Introdução

Caro(a) aluno(a),

No capítulo anterior aprendeu a tomar decisões utilizando as instruções if e else.

Agora imagine a seguinte situação.

O professor pede-lhe para escrever a palavra:

Bem-vindo

dez vezes no quadro.

Poderia escrever:

Bem-vindo

Bem-vindo

Bem-vindo

Bem-vindo

Bem-vindo

Bem-vindo

Bem-vindo

Bem-vindo

Bem-vindo

Bem-vindo

Mas isso seria cansativo.

Os computadores também enfrentam situações semelhantes.

Por vezes é necessário repetir a mesma tarefa várias vezes.

Para resolver esse problema utilizamos os **ciclos de repetição**.

Neste capítulo vai aprender a repetir tarefas automaticamente utilizando Python.

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Compreender o conceito de repetição;
- ✓ Utilizar a instrução for;
- ✓ Utilizar a função range();

- ✓ Repetir tarefas automaticamente;
- ✓ Criar programas mais eficientes.

1. O que é uma Repetição?

Observe algumas situações do dia-a-dia.

Na escola

1. Fazer chamada dos alunos;
2. Corrigir vários testes;
3. Registrar várias notas.

Na agricultura

1. Contar sacos de milho;
2. Registrar árvores plantadas.

No comércio

1. Contar produtos;
2. Registrar vendas.

Todas estas actividades envolvem repetições.

Os computadores conseguem efectuar estas tarefas automaticamente.

Figura 1 – Actividades repetitivas



Legenda: Muitas tarefas do quotidiano envolvem repetições.

2. O Ciclo for

Em Python utilizamos frequentemente a instrução:

for

A palavra inglesa **for** pode ser entendida como:

para cada

Primeiro exemplo

```
for i in range(5):  
    print("Olá!")
```

Resultado:

Olá!

Olá!

Olá!

Olá!

Olá!

Observe que a mensagem foi apresentada cinco vezes.

Como funciona?

A instrução:

```
range(5)
```

indica que a repetição deve ocorrer cinco vezes.

3. A Função range()

A função range() é muito utilizada com o ciclo for.

Observe:

```
for i in range(3):  
    print(i)
```

Resultado:

0

1

2

O que aconteceu?

Python começou em:

0

e terminou em:

2

porque o número final não é incluído.

Mais exemplos

```
for i in range(5):
```

```
    print(i)
```

Resultado:

0

1

2

3

4

Figura 2 – Contagem automática



Legenda: Os ciclos permitem efectuar contagens automaticamente.

4. Repetindo Mensagens

Uma das utilizações mais simples consiste em repetir mensagens.

Observe:

```
for i in range(10):
```

```
    print("Bem-vindo!")
```

Resultado:

Bem-vindo!

Bem-vindo!

Bem-vindo!

Bem-vindo!

Bem-vindo!

Bem-vindo!

Bem-vindo!

Bem-vindo!

Bem-vindo!

Bem-vindo!

Outro exemplo

```
for i in range(3):
```

```
    print("Estou a aprender Python.")
```

Resultado:

Estou a aprender Python.

Estou a aprender Python.

Estou a aprender Python.

5. Utilizando Variáveis no Ciclo

Podemos utilizar a variável de controlo.

Observe:

```
for i in range(5):
```

```
    print(i)
```

Resultado:

0

1

2

3

4

A variável:

i

recebe automaticamente diferentes valores.

Exemplo

```
for i in range(6):
```

```
    print("Número:", i)
```

Resultado:

Número: 0

Número: 1

Número: 2

Número: 3

Número: 4

Número: 5

6. Contando Alunos

Imagine uma turma com cinco alunos.

Podemos utilizar:

```
for i in range(5):
```

```
    print("Aluno", i + 1)
```

Resultado:

Aluno 1

Aluno 2

Aluno 3

Aluno 4

Aluno 5

Observe que utilizámos:

$i + 1$

para iniciar a contagem em 1.

Figura 3 – Lista de alunos



Legenda: Os ciclos podem ser utilizados para contar elementos.

7. Criando uma Tabuada Simples

Vamos utilizar repetições para criar uma tabuada.

Observe:

```
for i in range(1, 11):  
    print("2 x", i, "=", 2 * i)
```

Resultado:

2 x 1 = 2

2 x 2 = 4

2 x 3 = 6

...

2 x 10 = 20

Explicação

A instrução:

`range(1,11)`

produz os números:

1,2,3,4,5,6,7,8,9,10

Sabia que?

As tabuadas são um dos exemplos mais utilizados para aprender ciclos de repetição.

8. Utilizando Repetições no Quotidiano

Observe alguns exemplos.

Escola

for aluno in turma:

Processar todos os alunos.

Biblioteca

for livro in biblioteca:

Processar todos os livros.

Agricultura

for saco in colheita:

Contar todos os sacos.

Comércio

for produto in stock:

Verificar todos os produtos.

Figura 4 – Repetições em diferentes actividades



Legenda: As repetições estão presentes em muitos sistemas informáticos.

Erros Comuns

Erro 1 – Esquecer os dois pontos

Incorrecto:

```
for i in range(5)
```

Correcto:

```
for i in range(5):
```

Erro 2 – Problemas de indentação

Incorrecto:

```
for i in range(5):
```

```
print(i)
```

Correcto:

```
for i in range(5):
```

```
    print(i)
```

Erro 3 – Esquecer os parênteses

Incorrecto:

```
range 5
```

Correcto:

```
range(5)
```

Vamos praticar?

Actividade 1

Crie um programa que apresente:

Olá!

cinco vezes.

Actividade 2

Apresente os números:

0

1

2

3

4

utilizando um ciclo.

Actividade 3

Mostre o seu nome dez vezes.

Actividade 4

Crie a tabuada do número 3.

Desafio

Crie um programa que apresente os números de 1 até 20.

Dica:

Utilize:

`range(1,21)`

Vamos verificar o que aprendeu?

1. Qual instrução utilizamos para repetir tarefas?

- a) if
- b) print
- c) for
- d) input

2. O que faz `range(5)`?

- a) Repete infinitamente
- b) Produz cinco valores
- c) Mostra uma mensagem
- d) Recebe informações

3. Quantas vezes será apresentada a mensagem?

```
for i in range(3):
```

```
    print("Olá")
```

- a) 1
- b) 2
- c) 3
- d) 4

4. Qual é o primeiro valor produzido por range(5)?

- a) 1
- b) 5
- c) 0
- d) -1

5. Os ciclos ajudam a:

- a) Evitar repetições manuais
- b) Apagar programas
- c) Desligar computadores
- d) Criar erros

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Os ciclos permitem repetir tarefas automaticamente;
2. A instrução for é utilizada para criar repetições;
3. A função range() produz sequências de números;
4. É possível repetir mensagens e efectuar contagens;
5. Os ciclos tornam os programas mais eficientes.

Parabéns!

Agora os seus programas conseguem repetir tarefas automaticamente.

No próximo capítulo vamos aprender a **trabalhar com listas**, permitindo guardar vários valores numa única estrutura de dados.

CAPÍTULO 10 – TRABALHANDO COM LISTAS

Introdução

Caro(a) aluno(a),

Até agora aprendemos a guardar informações utilizando variáveis.

Por exemplo:

```
nome = "Amélia"
```

ou

```
idade = 14
```

Mas imagine a seguinte situação.

O professor pretende guardar os nomes de todos os alunos da turma.

Será necessário criar uma variável para cada aluno?

```
aluno1 = "Ana"
```

```
aluno2 = "Carlos"
```

```
aluno3 = "Paulo"
```

```
aluno4 = "Maria"
```

Isto funciona, mas rapidamente se torna difícil de organizar.

Felizmente, Python possui uma solução muito melhor.

Essa solução chama-se **lista**.

Neste capítulo vai aprender a guardar vários valores numa única estrutura chamada lista.

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Explicar o que é uma lista;
- ✓ Criar listas simples;
- ✓ Aceder aos elementos de uma lista;
- ✓ Adicionar novos elementos;
- ✓ Utilizar listas em programas simples.

1. O que é uma Lista?

Imagine uma turma com vários alunos.

Em vez de escrever cada nome em folhas separadas, pode criar uma lista contendo todos os nomes.

Na programação acontece exactamente o mesmo.

Uma lista permite guardar vários valores numa única variável.

Definição

Lista é uma estrutura utilizada para armazenar vários valores relacionados.

Exemplo

```
alunos = ["Ana", "Carlos", "Maria", "Paulo"]
```

Nesta lista existem quatro alunos.

Figura 1 – Lista de alunos



Legenda: Uma lista permite guardar vários elementos relacionados.

2. Criando a Primeira Lista

Observe:

```
provincias = ["Maputo", "Gaza", "Sofala"]
```

Esta lista contém três províncias.

Outro exemplo

```
disciplinas = ["Português", "Matemática", "TIC"]
```

Como identificar uma lista?

As listas utilizam:

```
[  
]
```

chamados colchetes.

Os elementos são separados por vírgulas.

Sabia que?

Uma lista pode conter dezenas, centenas ou até milhares de elementos.

3. Mostrar uma Lista

Podemos utilizar print().

Observe:

```
provincias = ["Maputo", "Gaza", "Sofala"]
```

```
print(provincias)
```

Resultado:

```
['Maputo', 'Gaza', 'Sofala']
```

Outro exemplo

```
frutas = ["Manga", "Banana", "Laranja"]
```

```
print(frutas)
```

Resultado:

```
['Manga', 'Banana', 'Laranja']
```

Figura 2 – Lista de elementos



Legenda: As listas podem guardar diferentes tipos de informações.

4. Acedendo aos Elementos

Uma lista funciona como uma fila organizada.

Cada elemento possui uma posição.

Observe:

```
provincias = ["Maputo", "Gaza", "Sofala"]
```

Posições:

Elemento Posição

Maputo 0

Gaza 1

Sofala 2

Exemplo

```
print(provincias[0])
```

Resultado:

Maputo

Outro exemplo

```
print(provincias[1])
```

Resultado:

Gaza

Atenção

Em Python a contagem começa em:

0

e não em 1.

5. Adicionando Novos Elementos

Podemos acrescentar elementos utilizando:

`append()`

Observe:

```
provincias = ["Maputo", "Gaza"]
```

```
provincias.append("Sofala")
```

```
print(provincias)
```

Resultado:

```
['Maputo', 'Gaza', 'Sofala']
```

Outro exemplo

```
alunos = ["Ana", "Carlos"]
```

```
alunos.append("Maria")
```

Resultado:

```
['Ana', 'Carlos', 'Maria']
```

Figura 3 – Crescimento da lista



Legenda: Novos elementos podem ser adicionados às listas.

6. Percorrendo uma Lista

Podemos utilizar um ciclo para mostrar todos os elementos.

Observe:

```
alunos = ["Ana", "Carlos", "Maria"]
```

```
for aluno in alunos:
```

```
    print(aluno)
```

Resultado:

Ana

Carlos

Maria

Como funciona?

O ciclo percorre todos os elementos da lista.

Um elemento de cada vez.

Exemplo com províncias

```
provincias = ["Maputo", "Gaza", "Sofala"]
```

```
for provincia in provincias:
```

```
    print(provincia)
```

Resultado:

Maputo

Gaza

Sofala

7. Utilizando Listas no Quotidiano

As listas são muito utilizadas em sistemas informáticos.

Escola

Lista de alunos:

alunos = ["Ana", "Paulo", "Maria"]

Biblioteca

Lista de livros:

livros = ["Matemática", "História", "Ciências"]

Agricultura

Lista de culturas:

culturas = ["Milho", "Feijão", "Mandioca"]

Comércio

Lista de produtos:

produtos = ["Arroz", "Açúcar", "Óleo"]

Figura 4 – Listas em diferentes áreas



Legenda: As listas são utilizadas em muitos sistemas do dia-a-dia.

8. Criando um Programa Completo

Vamos criar um pequeno programa.

```
disciplinas = ["Português", "Matemática", "TIC"]
```

```
print("Disciplinas da turma:")
```

```
for disciplina in disciplinas:
```

```
    print(disciplina)
```

Resultado:

Disciplinas da turma:

Português

Matemática

TIC

Parabéns!

Acabou de criar um programa que utiliza listas.

Erros Comuns

Erro 1 – Esquecer os colchetes

Incorrecto:

```
alunos = "Ana", "Carlos"
```

Correcto:

```
alunos = ["Ana", "Carlos"]
```

Erro 2 – Esquecer as aspas

Incorrecto:

```
alunos = [Ana, Carlos]
```

Correcto:

```
alunos = ["Ana", "Carlos"]
```

Erro 3 – Utilizar posição inexistente

Incorrecto:

```
provincias = ["Maputo"]
```

```
print(provincias[5])
```

Python apresentará um erro.

Vamos praticar?

Actividade 1

Crie uma lista contendo:

1. Maputo;
2. Gaza;
3. Inhambane.

Mostre a lista.

Actividade 2

Crie uma lista contendo três disciplinas.

Mostre todos os elementos.

Actividade 3

Crie uma lista com cinco nomes de colegas.

Apresente os nomes utilizando um ciclo for.

Actividade 4

Crie uma lista de frutas e adicione uma nova fruta utilizando append().

Desafio

Crie uma lista contendo as seguintes províncias:

1. Maputo;
2. Gaza;
3. Inhambane;
4. Sofala.

Depois apresente cada província numa linha diferente.

Vamos verificar o que aprendeu?

1. O que é uma lista?

- a) Um monitor
- b) Uma estrutura que guarda vários valores
- c) Um teclado
- d) Uma impressora

2. Quais símbolos identificam uma lista?

- a) ()
- b) {}
- c) []
- d) ""

3. Qual instrução cria uma lista?

- a)
`alunos = ["Ana", "Carlos"]`
- b)
`alunos = "Ana"`
- c)
`print("Ana")`
- d)
`input("Ana")`

4. Qual comando adiciona um elemento?

a)

`print()`

b)

`append()`

c)

`input()`

d)

`range()`

5. Qual é a posição do primeiro elemento?

a) 1

b) 2

c) 0

d) 10

Resumo do Capítulo

Neste capítulo aprendeu que:

1. Uma lista permite guardar vários valores;
2. As listas utilizam colchetes [];
3. Cada elemento possui uma posição;
4. A contagem começa em 0;
5. É possível adicionar novos elementos com `append()`;
6. Os ciclos podem percorrer listas automaticamente.

Parabéns!

Agora já consegue trabalhar com listas e organizar conjuntos de informações.

No próximo capítulo vamos aprender a **criar funções simples**, tornando os programas mais organizados e reutilizáveis.

CAPÍTULO 11 – O MEU PRIMEIRO PROJECTO EM PYTHON

Introdução

Caro(a) aluno(a),

Parabéns!

Chegou ao último capítulo deste manual.

Ao longo desta jornada aprendeu a:

1. Comunicar com o computador;
2. Criar programas simples;
3. Guardar informações;
4. Realizar cálculos;
5. Receber dados dos utilizadores;
6. Tomar decisões;
7. Repetir tarefas;
8. Trabalhar com listas.

Agora está na hora de juntar todos estes conhecimentos para construir um pequeno projecto.

Não se preocupe.

Vamos avançar passo a passo.

O que vai aprender?

Ao terminar este capítulo, deverá ser capaz de:

- ✓ Integrar vários conhecimentos aprendidos;
- ✓ Criar pequenos programas completos;
- ✓ Resolver problemas simples;
- ✓ Desenvolver confiança na programação.

1. O que é um Projecto?

Quando um programador cria uma aplicação, normalmente combina vários conhecimentos.

Por exemplo:

Um sistema escolar pode:

1. Receber nomes;
2. Guardar notas;
3. Calcular médias;
4. Verificar aprovações;
5. Mostrar resultados.

Tudo isto faz parte de um projecto.

Neste capítulo vamos criar pequenos projectos semelhantes.

Figura 1 – Construindo um projecto



Legenda: Um projecto combina vários conhecimentos de programação.

Projecto 1 – Ficha Electrónica do Aluno

Vamos criar um programa para registar informações de um aluno.

Passo 1 – Receber os dados

```
nome = input("Nome: ")
```

```
idade = input("Idade: ")
```

```
escola = input("Escola: ")
```

```
turma = input("Turma: ")
```

Passo 2 – Mostrar os dados

```
print("----- FICHA DO ALUNO -----")  
print("Nome:", nome)  
print("Idade:", idade)  
print("Escola:", escola)  
print("Turma:", turma)
```

Exemplo

Nome: Carlos

Idade: 14

Escola: Escola Secundária da Matola

Turma: 8ª Classe

Resultado:

----- FICHA DO ALUNO -----

Nome: Carlos

Idade: 14

Escola: Escola Secundária da Matola

Turma: 8ª Classe

Projecto 2 – Calculadora de Média

Agora vamos criar um programa para calcular médias.

Programa

```
nota1 = int(input("Primeira nota: "))
```

```
nota2 = int(input("Segunda nota: "))
```

```
media = (nota1 + nota2) / 2
```

```
print("Média:", media)
```

Exemplo

Entrada:

14

16

Resultado:

Média: 15.0

Figura 2 – Calculando médias



Legenda: Os computadores podem efectuar cálculos rapidamente.

Projecto 3 – Verificando Aprovação

Vamos combinar cálculo e decisão.

Programa

```
nota1 = int(input("Primeira nota: "))
```

```
nota2 = int(input("Segunda nota: "))
```

```
media = (nota1 + nota2) / 2
```

```
if media >= 10:
```

```
    print("Aprovado")
```

```
else:
```

```
    print("Reprovado")
```

Exemplo

Entrada:

12

14

Resultado:

Aprovado

Projecto 4 – Lista de Disciplinas

Agora vamos utilizar listas.

Programa

```
disciplinas = [
```

```
    "Português",
```

```
    "Matemática",
```

```
    "Ciências",
```

```
    "TIC"
```

```
]
```

```
for disciplina in disciplinas:
```

```
    print(disciplina)
```

Resultado:

Português

Matemática

Ciências

TIC

Figura 3 – Organização de informação



Legenda: As listas ajudam a organizar informações.

Projecto 5 – Saudação Personalizada

Vamos criar um programa amigável.

```
nome = input("Qual é o seu nome? ")
```

```
print("Olá,", nome)
```

```
print("Bem-vindo ao mundo da programação!")
```

Exemplo

Entrada:

Amélia

Resultado:

Olá, Amélia

Bem-vindo ao mundo da programação!

O que Aprendi Neste Manual?

Vamos recordar.

Capítulo 1

Aprendeu:

1. O que é programação;
2. O que é um programa.

Capítulo 2

Aprendeu:

1. O que é Python;
2. Como utilizar print().

Capítulo 3

Aprendeu:

1. Variáveis.

Capítulo 4

Aprendeu:

1. Operações matemáticas.

Capítulo 5

Aprendeu:

1. input().

Capítulo 6

Aprendeu:

- Como converter textos em números utilizando int();
- Como efectuar cálculos com dados introduzidos pelo utilizador;
- Como criar pequenas calculadoras;
- Como resolver problemas do quotidiano através da programação.

Capítulo 7

Aprende:

1. if e else.

Capítulo 8

Aprende:

1. for.

Capítulo 9

Aprende:

1. Listas.

Actividade Final

Crie um programa que:

1. Pergunte o nome;
2. Pergunte a idade;
3. Pergunte a escola;
4. Mostre todas as informações;
5. Apresente uma mensagem de boas-vindas.

Desafio Final

Crie um programa que:

1. Receba duas notas;
2. Calcule a média;
3. Informe se o aluno foi aprovado ou reprovado.

Figura 4 – O futuro programador



Legenda: Aprender programação é dar os primeiros passos para criar tecnologia.

Resumo do Capítulo

Neste capítulo:

1. Integrou os conhecimentos adquiridos;
2. Criou pequenos projectos completos;
3. Utilizou variáveis, decisões, listas e ciclos;
4. Desenvolveu confiança para continuar a aprender programação.

Parabéns!

Concluiu o seu primeiro manual de Introdução à Programação com Python. 🎉

Agora falta apenas prepararmos os elementos finais do manual (Glossário Geral, Respostas dos Exercícios, Bibliografia e Licença), e depois podemos gerar a versão final em Word.

GLOSSÁRIO

Algoritmo

Conjunto de passos ou instruções organizadas para resolver um problema ou realizar uma tarefa.

Aplicação

Programa criado para executar uma função específica, como uma calculadora, um navegador Web ou um sistema escolar.

CPU (Unidade Central de Processamento)

Componente responsável pela execução das instruções dos programas. É frequentemente chamada de "cérebro do computador".

Dados

Informações que podem ser armazenadas, processadas ou analisadas por um computador.

Erro

Problema que impede um programa de funcionar correctamente.

Função

Conjunto de instruções agrupadas para realizar uma tarefa específica.

Exemplo:

```
def saudacao():  
    print("Olá!")
```

Hardware

Conjunto dos componentes físicos do computador.

Exemplos:

1. Monitor;
2. Teclado;
3. Rato;
4. Impressora.

Input

Informação introduzida pelo utilizador ou por outro dispositivo.

Linguagem de Programação

Linguagem utilizada para comunicar com os computadores.

Exemplos:

1. Python;
2. Java;
3. JavaScript.

Lista

Estrutura utilizada para armazenar vários valores numa única variável.

Exemplo:

```
provincias = ["Maputo", "Gaza", "Sofala"]
```

Memória

Local onde os dados e programas são armazenados.

Output

Informação apresentada pelo computador após o processamento.

Programa

Conjunto de instruções que orienta o funcionamento do computador.

Programação

Processo de criação de programas.

Python

Linguagem de programação utilizada neste manual.

Variável

Espaço reservado para guardar informações.

Exemplo:

```
nome = "Amélia"
```

RESPOSTAS DOS EXERCÍCIOS

Capítulo 1 – Vamos Conversar sobre Programação

1. b
2. b
3. c
4. c
5. b

Capítulo 2 – Conhecendo o Python

1. b
2. b
3. c
4. c
5. b

Capítulo 3 – Guardando Informações com Variáveis

1. b
2. b
3. c
4. c
5. c

Capítulo 4 – Fazendo Contas com Python

1. c
2. a
3. c
4. b
5. b

Capítulo 5 – Conversando com o Utilizador

1. c
2. b
3. a
4. c
5. a

Capítulo 7 – Tomando Decisões com Python

1. c
2. c
3. b
4. c
5. c

Capítulo 8 – Repetindo Tarefas com Python

1. c
2. b
3. c
4. c
5. a

Capítulo 9 – Trabalhando com Listas

1. b
2. c
3. a
4. b
5. c

Capítulo 11 – O Meu Primeiro Projecto em Python

As actividades e desafios deste capítulo possuem respostas abertas, dependendo da solução criada pelo aluno.

BIBLIOGRAFIA

Obra principal utilizada na adaptação

Python para Todos

SEVERANCE, Charles. *Python para Todos: Explorando Dados com Python 3*.

Licença: Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0).

Disponível em:

[Python for Everybody \(PY4E\)](#)

Recursos complementares

Think Python

DOWNEY, Allen B. *Think Python: How to Think Like a Computer Scientist*.

Licença: Creative Commons Attribution-NonCommercial.

Python Software Foundation

Documentação oficial da linguagem Python:

[Python Documentation](#)

CRÉDITOS

Autor da obra original

Charles Severance

Autor do livro *Python para Todos: Explorando Dados com Python 3*, utilizado como principal referência para a elaboração deste manual.

Adaptação pedagógica

Este manual constitui uma adaptação educativa desenvolvida para:

1. Ensino Secundário;
2. II Ciclo (10 à 12.^a Classes);
3. Ensino à Distância;
4. Contexto moçambicano.

Foram efectuadas adaptações de:

1. linguagem;
2. exemplos;
3. exercícios;
4. actividades;
5. ilustrações sugeridas;
6. organização dos conteúdos.

LICENÇA DO REA DERIVADO

Recurso Educacional Aberto Derivado

Este manual foi desenvolvido com base na obra:

Python para Todos: Explorando Dados com Python 3, de Charles Severance.

A obra original encontra-se licenciada sob:

Creative Commons Attribution-NonCommercial-ShareAlike 3.0 (CC BY-NC-SA 3.0).

Por se tratar de uma adaptação da obra original, recomenda-se que este manual seja distribuído sob a mesma licença:

Creative Commons Atribuição-NãoComercial-PartilhaIgual 3.0
(CC BY-NC-SA 3.0)

Esta licença permite:

- ✓ copiar;
- ✓ distribuir;
- ✓ adaptar;
- ✓ remixar;
- ✓ reutilizar o material;

desde que:

1. seja atribuída a autoria da obra original;
2. não exista utilização comercial;
3. as obras derivadas sejam partilhadas sob a mesma licença.

Manual de Introdução à Programação com Python para o Ensino Secundário (10.^a–12.^a Classes), adaptado a partir da obra *Python para Todos: Explorando Dados com Python 3*, de Charles Severance, licenciada sob CC BY-NC-SA 3.0.

GUIA DE REUTILIZAÇÃO DO RECURSO

Este recurso educacional aberto foi preparado para facilitar a reutilização por professores, estudantes e instituições de ensino.

O material pode ser utilizado em diferentes contextos, tais como:

- aulas presenciais de Informática;
- ensino à distância;
- plataformas digitais como Moodle;
- formação inicial de professores;
- cursos introdutórios de programação;
- aprendizagem autónoma dos estudantes.

Por estar disponível em formato editável, este recurso pode ser adaptado conforme diferentes necessidades pedagógicas, respeitando sempre a licença Creative Commons estabelecida.

A reutilização de Recursos Educacionais Abertos contribui para democratizar o acesso ao conhecimento e fortalecer práticas colaborativas de ensino.